LangChain Notes

Module 1: Introduction to LangChain

---

1. What is LangChain?

Definition

LangChain is an open-source framework used to build applications powered by **Large Language Models (LLMs)**. It helps developers connect LLMs with external data, tools, APIs, databases, and workflows.

In simple words: LangChain makes it easier to build AI applications using LLMs.

Why is it Important?

Without LangChain, developers must manually write code to:

- Connect LLMs with APIs
- Read documents
- Store conversation history
- Call external tools
- Build workflows

LangChain provides ready-made modules to handle these tasks.

Key Points

- Open-source framework
- Works with many LLMs
- Connects LLMs to external data
- Supports AI agents and automation
- Easy integration with APIs and databases

Example

Without LangChain:

User → LLM → Response

With LangChain:

User → LangChain → LLM + Database + API + Tools → Response

Remember

LangChain = Framework for building LLM applications

---

2. Why LangChain?

Definition

LangChain is used to simplify the development of AI applications by providing reusable components and workflows.

Why Do We Need It?

Building LLM applications from scratch is difficult because developers need to:

- Manage prompts
- Connect APIs
- Handle memory
- Retrieve documents
- Use external tools
- Build workflows

LangChain solves these problems.

Advantages

- Faster development
- Less coding
- Reusable components
- Easy integration
- Supports multiple LLM providers
- Scalable applications

Real-Life Example

Instead of writing hundreds of lines of code to build a chatbot, LangChain provides ready-to-use components.

Remember

LangChain saves development time and reduces complexity.

---

3. Problems LangChain Solves

Definition

LangChain solves common challenges faced while developing LLM-based applications.

Problems

1. No Access to External Data

LLMs only know what they learned during training.

Solution: Connect databases, PDFs, websites, and documents.

---

2. No Memory

Basic LLMs forget previous conversations.

Solution: Add conversation memory.

---

3. Cannot Use Tools

LLMs cannot directly:

- Search Google

- Call APIs
- Use calculators
- Access databases

Solution: Tool integration.

---

4. Complex Workflows

Managing multiple LLM calls becomes difficult.

Solution: Build structured workflows.

---

5. Prompt Management

Writing prompts repeatedly is inefficient.

Solution: Prompt templates.

Summary

LangChain provides:

- External data access
- Memory
- Tool calling
- Workflow management
- Prompt management

---

4. LangChain Architecture

Definition

LangChain architecture shows how different components work together to process user requests.

Basic Architecture

User

Prompt

LLM

Tools / Documents / APIs

Final Response

Flow

1. User asks a question.
2. Prompt is prepared.
3. LLM processes the prompt.
4. LangChain retrieves data or uses tools if needed.
5. LLM generates the final response.

Key Components

- User Input
- Prompt
- LLM
- Tools
- External Data
- Output

Remember

LangChain acts as a bridge between the user, the LLM, and external resources.

---

5. LangChain Ecosystem

Definition

The LangChain ecosystem consists of tools and libraries that help build complete AI applications.

Main Parts

1. LangChain

Core framework for building LLM applications.

---

2. LangGraph

Builds multi-agent and workflow-based AI applications.

---

3. LangSmith

Used for:

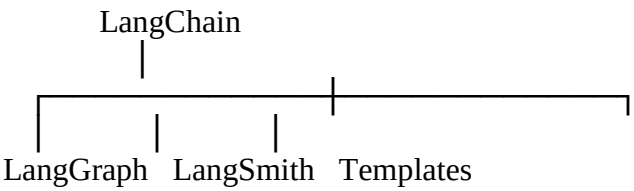
- Debugging
- Monitoring
- Evaluation
- Tracing applications

---

4. Templates

Ready-made application templates.

Diagram



Remember

- **LangChain → Build applications**
- **LangGraph → Agent workflows**
- **LangSmith → Debug & monitor**

---

6. LangChain vs Traditional LLM Applications

Traditional LLM	LangChain
Only prompt and response	Complete application framework
No memory	Supports memory
No tools	Tool integration
No document retrieval	Supports retrieval
Manual coding	Ready-made components
Difficult workflows	Easy workflow management
Limited functionality	Highly scalable

Example

Traditional:

Question → LLM → Answer

LangChain:

Question
↓
Prompt
↓
LLM
↓
Database
API
Tools
Memory
↓
Final Answer

Remember

Traditional LLM = Only text generation
LangChain = Complete AI application development

---

7. LangChain v1 Overview

Definition

LangChain v1 is the latest version of LangChain, designed to make building LLM applications simpler, faster, and more modular.

Features

- Simplified API
- Better performance
- Improved agent support
- Easier integration
- Better developer experience
- Cleaner architecture

Benefits

- Less code
- Easier maintenance
- Better scalability
- Improved reliability

Remember

LangChain v1 focuses on simplicity, modularity, and performance.

---

8. Use Cases

Common Applications

Chatbots

Customer support and virtual assistants.

Question Answering

Answer questions from PDFs, websites, or documents.

AI Agents

AI that can use tools and perform tasks automatically.

Document Analysis

Summarize, search, and extract information.

Code Generation

Generate or explain programming code.

Content Creation

Generate:

- Emails
- Blogs
- Reports
- Social media posts

Data Analysis

Analyze structured and unstructured data.

Real-World Examples

- Customer support bots
- Resume analyzers
- AI coding assistants

- PDF chatbots
- Research assistants
- Healthcare assistants
- Financial assistants

---

9. Installation

Step 1: Create a Virtual Environment (Recommended)

python -m venv .venv

Activate it:

Windows

.venv\Scripts\activate

Linux/macOS

source .venv/bin/activate

Step 2: Install LangChain

pip install langchain

Or with **uv**:

uv add langchain

Install Common Packages

pip install langchain-openai
pip install langchain-community
pip install python-dotenv

Check Installation

import langchain
print(langchain.__version__)

Remember

Always use a virtual environment to avoid dependency conflicts.

---

10. First LangChain Application

Example

```
from langchain.chat_models import init_chat_model

llm = init_chat_model(
   "gpt-4.1-mini",
   model_provider="openai"
)

response = llm.invoke("What is LangChain?")
print(response.content)
```

How It Works

1. Import LangChain.
2. Initialize the chat model.
3. Send a prompt using invoke().
4. Receive the AI-generated response.

Output Example

LangChain is an open-source framework for building applications powered by Large Language Models (LLMs).

Module 2: LangChain Core Concepts

---

1. Models

Definition

A **Model** is the AI engine that understands user input and generates responses. LangChain provides a common interface to work with different AI models.

In simple words: A model is the "brain" that processes your prompt and gives an answer.

Why is it Used?

- Generate text
- Answer questions
- Summarize documents
- Translate languages
- Write code

Key Points

- Main component of every LangChain application.
- Supports multiple AI providers.
- Easy to switch between models.
- Same code works with different providers.

Examples of Models

- OpenAI GPT
- Google Gemini
- Anthropic Claude
- Groq
- Ollama (Local Models)

Remember

Model = AI Brain

---

2. Chat Models

Definition

A **Chat Model** is a type of language model designed for conversations. It accepts messages (user, system, assistant) and returns a conversational response.

Why is it Used?

- Build chatbots
- Virtual assistants
- Customer support
- AI conversations

Key Points

- Uses message-based input.
- Maintains conversation format.
- Supports roles like System, User, and Assistant.
- Most modern LLM applications use chat models.

Example

User: What is AI?
Assistant: AI stands for Artificial Intelligence.

Remember

Chat Model = Conversation-based AI

---

3. Language Models

Definition

A **Language Model (LLM)** predicts and generates text based on the given input.

Why is it Used?

- Text generation
- Story writing
- Summarization
- Translation

- Code generation

Key Points

- Works with plain text.
- Generates the next likely words.
- Understands context.
- Can answer a wide variety of questions.

Example

Input:
Explain Machine Learning.

Output:
Machine Learning is a branch of Artificial Intelligence...

Difference

Chat Model	Language Model
Uses messages	Uses plain text
Conversation-based	Text-based
Chatbots	General text generation

Remember

Language Model = Text Generator
Chat Model = Conversation Generator

---

4. Messages

Definition

Messages are the inputs and outputs exchanged between the user and the chat model.

Types of Messages

1. System Message

Gives instructions to the AI.

Example:

You are a helpful assistant.

---

2. Human (User) Message

Question or request from the user.

Example:

Explain LangChain.

---

3. AI (Assistant) Message

Response generated by the AI.

Example:

LangChain is an open-source framework...

Flow

System Message
↓
User Message
↓
AI Response

Remember

- **System → Rules**
- **User → Question**
- **AI → Answer**

---

5. Prompt Templates

Definition

A **Prompt Template** is a reusable prompt with placeholders for dynamic values.

Why is it Used?

Instead of writing prompts repeatedly, we create one template and replace variables with actual values.

Example

Template:

Explain {topic} in simple words.

Input:

topic = "LangChain"

Final Prompt:

Explain LangChain in simple words.

Advantages

- Reusable
- Dynamic
- Cleaner code
- Easier maintenance

Remember

Prompt Template = Prompt + Variables

---

6. Output Parsers

Definition

An **Output Parser** converts the model's output into a structured format.

Why is it Used?

Sometimes we need output as:

- JSON
- List
- Dictionary
- Table

instead of plain text.

Example

Without Parser

Name: Rahul
Age: 22

With Parser

```
{
  "name": "Rahul",
  "age": 22
}
```

Advantages

- Structured output
- Easy processing
- Better automation

Remember

Output Parser = Formats AI output

---

7. Runnables

Definition

A **Runnable** is any object that can receive input, perform an operation, and produce output.

Why is it Used?

It provides a common way to execute different LangChain components.

Examples

- Prompt Template
- Chat Model
- Output Parser
- Chains

All of these are runnables.

Flow

Input
↓
Runnable
↓
Output

Advantages

- Easy composition
- Reusable
- Flexible execution

Remember

Runnable = Executable component

---

8. LangChain Expression Language (LCEL)

Definition

LCEL (LangChain Expression Language) is a simple syntax used to connect multiple runnables into a workflow.

Why is it Used?

Instead of manually calling each component, LCEL lets you connect them using the **pipe (|) operator**.

Example

Prompt

Chat Model

Output Parser

Using LCEL:

prompt | model | parser

Advantages

- Less code
- Readable
- Easy to build pipelines
- Reusable workflows

Remember

LCEL = Connect components using |

---

9. Invoke

Definition

invoke() executes a runnable for **one input** and returns **one output**.

Syntax

result = runnable.invoke(input)

Example

model.invoke("What is AI?")

Output

Artificial Intelligence is...

When to Use?

- Single question
- Single prompt
- One request at a time

Remember

invoke() = One Input → One Output

---

10. Batch

Definition

batch() executes the same runnable on **multiple inputs** at once.

Syntax

runnable.batch([input1, input2, input3])

Example

```
[
 "Explain AI",
 "Explain ML",
 "Explain NLP"
]
```

Output

AI explanation
ML explanation
NLP explanation

Advantages

- Faster processing
- Multiple requests together
- Better performance

Remember

batch() = Many Inputs → Many Outputs

---

11. Stream

Definition

stream() returns the response **piece by piece** instead of waiting for the complete answer.

Why is it Used?

Provides real-time responses, improving user experience.

Example

Instead of:

Hello! How can I help you today?

You receive:

Hello...
How...
can...
I...
help...
you...
today?

Uses

- Chatbots
- Live AI responses
- Interactive applications

Remember

stream() = Real-time output

---

12. Async Execution

Definition

Async Execution allows multiple tasks to run concurrently without waiting for each one to finish before starting the next.

Why is it Used?

Improves speed and efficiency when working with multiple LLM requests or APIs.

Example

Instead of:

Task 1 → Finish
Task 2 → Finish
Task 3 → Finish

Async execution allows:

Task 1
Task 2
Task 3
Running Together

Advantages

- Faster execution
- Better resource utilization
- Handles many requests efficiently

Common Async Methods

- ainvoke() → Async version of invoke()
- abatch() → Async version of batch()
- astream() → Async version of stream()

Remember

Async = Multiple tasks running concurrently

Module 3: Prompt Templates

---

1. Prompt Templates

Definition

A **Prompt Template** is a reusable prompt that contains **placeholders (variables)**. The placeholders are replaced with actual values when the prompt is executed.

In simple words: Write the prompt once and reuse it with different inputs.

Why is it Used?

- Avoid writing the same prompt repeatedly.
- Make prompts dynamic.
- Keep code clean and reusable.

Example

Template:

Explain {topic} in simple words.

Input:

topic = "LangChain"

Final Prompt:

Explain LangChain in simple words.

Advantages

- Reusable
- Easy to maintain
- Dynamic input
- Less code

Remember

Prompt Template = Prompt + Variables

---

2. Chat Prompt Templates

Definition

A **Chat Prompt Template** is used to create prompts for **chat models**. It combines different message types like **System**, **Human**, and **AI** messages.

Why is it Used?

- Build conversational AI.
- Organize chat messages.
- Give instructions and examples.

Structure

System Message
↓
Human Message
↓
AI Response

Example

System: You are a helpful assistant.

Human: Explain AI.

AI: Artificial Intelligence is...

Remember

Chat Prompt Template = Collection of chat messages

---

3. System Message

Definition

A **System Message** gives instructions or rules to the AI before the conversation begins.

Why is it Used?

- Define AI behavior.
- Set tone or role.
- Control response style.

Example

You are a Python expert.

or

Answer in simple English.

Key Points

- Sent first.
- Not visible to the end user.
- Controls the AI's behavior.

Remember

System Message = Instructions for AI

---

4. Human Message

Definition

A **Human Message** represents the user's question or request.

Example

Explain Machine Learning.

Key Points

- Comes from the user.
- Contains the query.
- AI responds to it.

Flow

Human → AI

Remember

Human Message = User's Input

---

5. AI Message

Definition

An **AI Message** is the response generated by the AI model.

Example

Machine Learning is a branch of Artificial Intelligence...

Key Points

- Generated by the model.
- Can be stored as conversation history.
- Used to maintain context.

Flow

Human → AI Message

Remember

AI Message = Model's Response

---

6. Placeholder Messages

Definition

A **Placeholder Message** reserves space for messages that will be added later, such as conversation history.

In simple words: It is an empty slot that gets filled during execution.

Why is it Used?

- Store chat history.
- Build dynamic conversations.
- Reuse the same prompt structure.

Example

Template:

System: You are a helpful assistant.

{chat_history}

Human: {question}

During execution:

System: You are a helpful assistant.

User: Hi
AI: Hello!

Human: What is LangChain?

Advantages

- Supports conversation memory.

- Makes prompts flexible.
- Keeps templates clean.

Remember

Placeholder = Space for future messages

---

7. Partial Prompts

Definition

A **Partial Prompt** is a prompt template where some variables are filled in advance, while others are filled later.

Why is it Used?

- Avoid repeating fixed values.
- Reuse common prompt parts.
- Simplify prompt creation.

Example

Template:

Explain {topic} in {language}.

Partial Prompt:

language = English

Later:

topic = LangChain

Final Prompt:

Explain LangChain in English.

Advantages

- Reusable
- Cleaner code
- Less repetition

Remember

Partial Prompt = Some variables filled now, others later

---

8. Dynamic Prompts

Definition

A **Dynamic Prompt** changes its content based on user input, data, or program logic.

Why is it Used?

- Create personalized prompts.
- Adapt to different situations.
- Build smarter AI applications.

Example

If user selects:

Topic = Python

Prompt becomes:

Explain Python.

If user selects:

Topic = Java

Prompt becomes:

Explain Java.

Advantages

- Flexible
- Personalized
- Works for many scenarios

Remember

Dynamic Prompt = Prompt changes automatically

---

9. Prompt Composition

Definition

Prompt Composition is the process of combining multiple prompt parts into one complete prompt.

Why is it Used?

- Build complex prompts.
- Organize instructions clearly.
- Reuse prompt sections.

Example

Components:

System:
You are a helpful teacher.

Human:
Explain {topic}.

Format:
Use bullet points.

Combined Prompt:

You are a helpful teacher.

Explain LangChain.

Use bullet points.

Benefits

- Modular design
- Easy maintenance
- Reusable prompt sections
- Cleaner prompt structure

Remember

Prompt Composition = Combine multiple prompt pieces into one prompt

Module 4: Models

---

1. OpenAI

Definition

OpenAI provides powerful Large Language Models (LLMs) such as GPT that can understand and generate human-like text.

In simple words: OpenAI offers AI models for chat, coding, writing, summarization, and more.

Why is it Used?

- Chatbots
- Content generation
- Code generation
- Translation
- Summarization

Key Points

- High-quality responses
- Easy integration with LangChain
- Supports chat and reasoning models
- API-based service

Example Models

- GPT-4.1
- GPT-4.1 Mini
- GPT-5 (depending on availability)

Remember

OpenAI = Powerful cloud-based AI models

---

2. Groq

Definition

Groq provides extremely fast inference for open-source LLMs.

In simple words: Groq is known for generating AI responses very quickly.

Why is it Used?

- Fast AI applications
- Chatbots
- Real-time assistants
- Low latency

Key Points

- Very fast inference
- API compatible with LangChain

- Supports popular open-source models
- Cloud-based

Remember

Groq = Speed

---

3. Gemini

Definition

Gemini is Google's family of AI models that can understand and generate text, images, audio, and more.

Why is it Used?

- Chatbots
- Content generation
- Multimodal AI
- Coding assistance

Key Points

- Developed by Google
- Supports multimodal inputs
- Integrates with LangChain
- Cloud-based

Remember

Gemini = Google's AI model

---

4. Anthropic Claude

Definition

Claude is a family of AI models developed by Anthropic, designed to provide safe, helpful, and reliable responses.

Why is it Used?

- Long document analysis
- Chatbots
- Coding
- Research

Key Points

- Strong reasoning
- Large context window
- Focus on AI safety
- API integration

Remember

Claude = Safe and reliable AI

---

5. Ollama

Definition

Ollama allows you to run open-source AI models locally on your own computer.

Why is it Used?

- Offline AI
- Privacy
- Local development
- No internet required (after downloading models)

Key Points

- Runs locally
- Supports models like Llama, Mistral, Gemma, etc.
- Free to use
- Integrates with LangChain

Remember

Ollama = Local AI models

---

6. Hugging Face

Definition

Hugging Face is a platform that provides thousands of open-source AI models and datasets.

Why is it Used?

- Access pre-trained models
- NLP tasks
- Computer vision
- Model sharing

Key Points

- Huge model library
- Open-source community
- Easy LangChain integration
- Supports many ML tasks

Remember

Hugging Face = Hub for AI models

---

7. Model Parameters

Definition

Model parameters are **settings** that control how an AI model generates responses.

Common Parameters

- Temperature
- Max Tokens
- Top P
- Stop Sequences

Why are They Important?

They help control:

- Creativity
- Response length
- Randomness
- Output format

Remember

Parameters = AI behavior settings

---

8. Temperature

Definition

Temperature controls how creative or random the AI's responses are.

Range

0.0 ----------> 2.0
Less Random More Random

Values

Low Temperature (0–0.3)

- More accurate
- Predictable
- Consistent

Used for:

- Coding
- Mathematics
- Facts

Medium Temperature (0.5–0.7)

- Balanced creativity

Used for:

- Chatbots
- General Q&A

High Temperature (0.8–1.5+)

- More creative
- More diverse

Used for:

- Story writing
- Brainstorming
- Creative content

Remember

Lower = More accurate
Higher = More creative

---

9. Max Tokens

Definition

Max Tokens sets the maximum number of tokens the model can generate in its response.

Why is it Used?

- Control response length
- Reduce API cost
- Prevent overly long answers

Example

Max Tokens = 50

The AI will stop after generating approximately 50 tokens.

Remember

Max Tokens = Maximum response length

---

10. Top P

Definition

Top P (Nucleus Sampling) controls randomness by selecting words from the most probable group whose cumulative probability reaches **P**.

Example

Top P = 1.0

Uses all possible words.

Top P = 0.8

Uses only the most likely words until their combined probability reaches 80%.

Key Points

- Lower Top P → More focused output
- Higher Top P → More diverse output

Remember

Top P controls word selection diversity.

---

11. Stop Sequences

Definition

A **Stop Sequence** tells the model where it should stop generating text.

Example

Stop Sequence:

###

Output:

Answer: AI is...
###

The model stops when it reaches ###.

Uses

- Prevent extra text
- Control output format
- End responses automatically

Remember

Stop Sequence = Generation stopping point

Module 5: Output Parsers

---

1. StrOutputParser

Definition

StrOutputParser converts the model's output into a simple text string.

In simple words: It returns plain text without extra formatting.

Why is it Used?

- Simple responses
- Chatbots
- Text generation

Example

Model Output:

LangChain is an open-source framework.

After Parsing:

LangChain is an open-source framework.

(String)

Remember

StrOutputParser = Plain text output

---

2. JSON Output Parser

Definition

A **JSON Output Parser** converts the AI response into **JSON format**.

Why is it Used?

- APIs
- Structured data
- Automation

Example

Output

```
{
  "name": "Rahul",
  "age": 22
}
```

Advantages

- Easy to process
- Machine-readable
- Structured format

Remember

JSON Parser = JSON output

---

3. Structured Output

Definition

Structured Output means the model returns information in a predefined format instead of free-form text.

Common Formats

- JSON
- Dictionary
- List
- Table

Example

Instead of

Rahul is 22 years old.

Return

```
{
  "name": "Rahul",
  "age": 22
}
```

Advantages

- Consistent
- Easy parsing
- Better automation

Remember

Structured Output = Organized data

---

4. Pydantic Output Parser

Definition

A **Pydantic Output Parser** validates and converts model output into a **Pydantic object**.

Why is it Used?

- Data validation
- Type checking
- Error reduction

Example

Pydantic Model

```
class Student:
   name: str
   age: int
```

Output

Student(name="Rahul", age=22)

Advantages

- Strong validation
- Clean objects
- Reliable data

Remember

Pydantic Parser = Validated structured output

---

5. Retry Parser

Definition

A **Retry Parser** asks the model to generate the output again if parsing fails.

Why is it Used?

- Handle invalid output
- Improve reliability
- Reduce parsing errors

Example

First Output

Invalid JSON

Retry

```
{
  "name": "Rahul"
}
```

Remember

Retry Parser = Try again if parsing fails

---

6. Output Fixing Parser

Definition

An **Output Fixing Parser** automatically fixes formatting errors in the model's output without asking the model to regenerate the response.

Why is it Used?

- Correct invalid JSON
- Fix formatting mistakes
- Improve automation

Example

Invalid Output

{name: Rahul}

Fixed Output

```
{
  "name": "Rahul"
}
```

Advantages

- Automatic correction
- Saves API calls
- Faster processing

Remember

Output Fixing Parser = Automatically fixes formatting errors

---

7. Custom Output Parser

Definition

A **Custom Output Parser** is a parser created by the developer for specific output requirements.

Why is it Used?

- Custom data formats
- Business-specific logic
- Special validation

Example

Required Output

Name : Rahul
Marks : 95
Grade : A

A custom parser extracts:

```
{
   "name": "Rahul",
   "marks": 95,
   "grade": "A"
}
```

Advantages

- Flexible
- Fully customizable
- Works with any format

Remember

Custom Output Parser = Developer-defined parser

Module 6: LCEL (LangChain Expression Language)

---

1. What is LCEL?

Definition

LCEL (LangChain Expression Language) is a simple way to connect LangChain components (called **Runnables**) into a workflow using the **pipe (|) operator**.

In simple words: LCEL lets you build AI pipelines with less code.

Why is it Used?

- Connect multiple components easily
- Reduce boilerplate code
- Create reusable workflows
- Build complex AI pipelines

Example

Instead of calling each component separately:

Prompt → Model → Parser

LCEL:

prompt | model | parser

Advantages

- Simple syntax
- Readable code
- Reusable pipelines
- Easy debugging

Remember

LCEL = Connect Runnables using |

---

2. Runnable Interface

Definition

A **Runnable** is any object that accepts **input**, performs an operation, and returns **output**.

Examples

- Prompt Template

- Chat Model
- Output Parser
- Chain

Flow

Input
↓
Runnable
↓
Output

Remember

Runnable = Executable component

---

3. RunnableSequence

Definition

A **RunnableSequence** executes multiple runnables **one after another**.

Flow

Prompt
↓
Model
↓
Parser

Example

prompt | model | parser

Uses

- Chatbots

- Q&A systems
- RAG pipelines

Remember

RunnableSequence = Sequential execution

---

4. RunnableLambda

Definition

A **RunnableLambda** allows you to use a **Python function** as a runnable.

Why is it Used?

- Add custom logic
- Modify input/output
- Perform calculations

Example

```
def uppercase(text):
    return text.upper()
```

The function becomes part of the pipeline.

Remember

RunnableLambda = Custom Python function

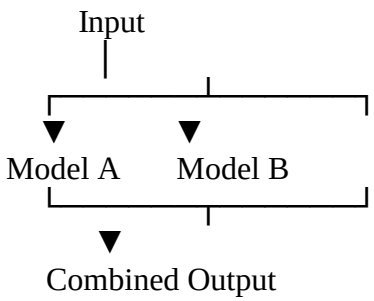
---

5. RunnableParallel

Definition

A **RunnableParallel** runs multiple runnables **at the same time** and combines their outputs.

Flow



Advantages

- Faster execution
- Multiple results together
- Better performance

Remember

RunnableParallel = Parallel execution

---

6. RunnableMap

Definition

A **RunnableMap** applies different runnables to the same input and returns all results as a dictionary.

Example

Input:

LangChain

Output:

```
{
 "summary": "...",
 "keywords": "...",
 "translation": "..."
}
```

Uses

- Multiple analyses
- Feature extraction
- Multi-output processing

Remember

RunnableMap = One input → Multiple outputs

---

7. RunnablePassthrough

Definition

A **RunnablePassthrough** passes the input to the next runnable **without changing it**.

Why is it Used?

- Keep original input
- Add extra data
- Reuse existing values

Flow

Input
  |
(No Change)
  |
Output

Remember

RunnablePassthrough = Forward input unchanged

---

8. Pipe Operator (|)

Definition

The **Pipe Operator (|)** connects multiple runnables into a pipeline.

Example

prompt | model | parser

Flow

Prompt

Model

Parser

Advantages

- Cleaner code
- Easy composition
- Readable pipelines

Remember

| = Connect components

---

9. Assign

Definition

assign() adds **new fields** to the current data while keeping the original data unchanged.

Example

Input

```
{
 "question": "What is AI?"
}
```

After Assign

```
{
 "question": "What is AI?",
 "length": 11
}
```

Uses

- Add metadata
- Compute extra values
- Extend pipeline data

Remember

Assign = Add new data

---

10. Bind

Definition

bind() permanently attaches fixed parameters to a runnable.

Why is it Used?

- Avoid repeating settings
- Fix model parameters
- Reuse configuration

Example

Instead of:

model.invoke(prompt, temperature=0)

Bind once:

model.bind(temperature=0)

Now every call uses the same temperature.

Remember

Bind = Attach fixed settings

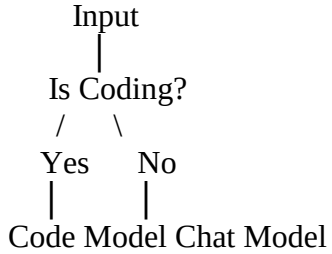
---

11. Branching

Definition

Branching executes different runnables based on a condition.

Example



Uses

- Decision making
- Dynamic workflows
- Smart routing

Remember

Branching = Choose execution path

---

12. Streaming Pipelines

Definition

A **Streaming Pipeline** returns output **token by token** while the pipeline is still running.

Why is it Used?

- Faster user experience
- Live responses
- Chat applications

Example

Instead of waiting:

Hello! How can I help you today?

You receive:

Hello...
How...
can...
I...
help...
you...
today?

Uses

- Chatbots
- AI assistants
- Live applications

Remember

Streaming Pipeline = Real-time output

Module 7: Chains

---

1. What is a Chain?

Definition

A **Chain** is a sequence of one or more LangChain components that work together to complete a task.

In simple words: A chain connects multiple steps so the output of one step becomes the input of the next.

Flow

Input
↓
Prompt
↓
Model
↓
Parser
↓
Output

Why is it Used?

- Automate workflows
- Reduce repetitive code
- Build AI applications

Remember

Chain = Connected workflow

---

2. Sequential Chains

Definition

A **Sequential Chain** executes each step **one after another**.

Flow

Input
↓
Summarize
↓
Translate
↓
Output

Uses

- Document processing
- Translation
- Summarization

Advantages

- Simple
- Easy to understand
- Reliable

Remember

Sequential Chain = One step after another

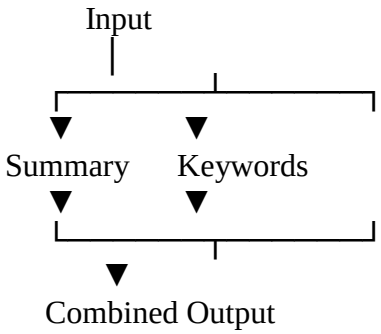
---

3. Parallel Chains

Definition

A **Parallel Chain** executes multiple chains **simultaneously**.

Flow



Advantages

- Faster execution
- Multiple results together
- Better efficiency

Remember

Parallel Chain = Multiple tasks at once

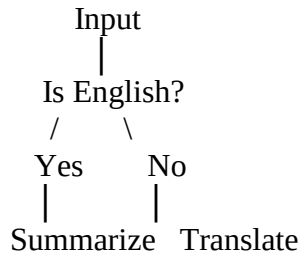
---

4. Conditional Chains

Definition

A **Conditional Chain** executes different chains depending on a condition.

Example



Uses

- Decision making
- Dynamic workflows
- Smart routing

Remember

Conditional Chain = If-Else logic

---

5. Transform Chains

Definition

A **Transform Chain** modifies or transforms data before passing it to the next step.

Example

Input

hello world

Transform

HELLO WORLD

Then send it to the model.

Uses

- Text cleaning
- Formatting
- Data preprocessing

Remember

Transform Chain = Modify data

---

6. Custom Chains

Definition

A **Custom Chain** is a chain created by the developer to perform application-specific tasks.

Why is it Used?

- Business logic
- Custom workflows
- Specialized processing

Example

Resume
↓
Extract Skills
↓
Calculate Score
↓
Generate Feedback

Advantages

- Flexible
- Fully customizable
- Fits application requirements

Remember

Custom Chain = Developer-designed workflow

---

7. Chain Composition

Definition

Chain Composition is the process of combining multiple chains into one larger workflow.

Example

Input
↓
Summary Chain
↓
Translation Chain
↓
Sentiment Chain
↓
Output

Advantages

- Modular design
- Reusable chains
- Easy maintenance
- Scalable applications

Remember

Chain Composition = Combine multiple chains into one workflow

Module 8: Memory

---

1. Why Memory?

Definition

Memory allows an AI application to remember previous interactions during a conversation.

In simple words: Memory helps the AI remember what was discussed earlier.

Why is it Needed?

Without Memory:

User: My name is Rahul.

User: What is my name?

AI: I don't know.

With Memory:

User: My name is Rahul.

User: What is my name?

AI: Your name is Rahul.

Advantages

- Maintains conversation context
- Better user experience
- Personalized responses
- Supports long conversations

Remember

Memory = Conversation History

---

2. Conversation Buffer Memory

Definition

Conversation Buffer Memory stores the **entire conversation history**.

How It Works

User: Hi
AI: Hello

User: What is AI?
AI: Artificial Intelligence...

User: Explain ML.

The complete conversation is stored.

Advantages

- Full context
- Simple to use
- Good for short conversations

Disadvantages

- Memory grows continuously
- More tokens used
- Higher API cost

Remember

Buffer Memory = Stores everything

---

3. Window Memory

Definition

Window Memory stores only the **last N conversations** instead of the entire chat history.

Example

Window Size = 3

Stored:

Message 3
Message 4
Message 5

Older messages are removed.

Advantages

- Saves tokens
- Faster
- Lower cost

Disadvantages

- Older context is lost

Remember

Window Memory = Last N messages

---

4. Summary Memory

Definition

Summary Memory stores a **summary** of previous conversations instead of the complete history.

Example

Original Conversation

20 messages

Stored Summary

The user is learning LangChain and asks beginner questions.

Advantages

- Saves tokens
- Supports long conversations
- Keeps important information

Remember

Summary Memory = Conversation summary

---

5. Token Buffer Memory

Definition

Token Buffer Memory stores conversation history based on the **maximum number of tokens**, not the number of messages.

Example

Limit:

1000 Tokens

When the limit is reached, the oldest content is removed.

Advantages

- Controls token usage
- Better API cost management

- Efficient for long chats

Remember

Token Buffer = Token-based memory

---

6. Entity Memory

Definition

Entity Memory remembers important entities such as **people, places, companies, and objects** mentioned during the conversation.

Example

User: My manager is Priya.

Later...

User: Who is my manager?

AI: Your manager is Priya.

Stores

- Names
- Companies
- Cities
- Products
- Organizations

Remember

Entity Memory = Important names and entities

---

7. Combined Memory

Definition

Combined Memory combines two or more memory types into one system.

Example

Conversation Buffer
+
Summary Memory
+
Entity Memory

Advantages

- Better context
- More flexible
- Improved conversations

Remember

Combined Memory = Multiple memory types together

---

8. Custom Memory

Definition

A **Custom Memory** is a memory system created by the developer for specific application needs.

Why is it Used?

- Business applications
- Healthcare
- Banking
- CRM systems

Example

A shopping assistant remembers:

Favorite Brand
Previous Orders
Delivery Address

Advantages

- Fully customizable
- Fits business requirements
- Flexible implementation

Remember

Custom Memory = Developer-defined memory

Module 9: Document Loaders

---

1. What is a Document Loader?

Definition

A **Document Loader** loads data from different file types and converts it into a format that LangChain can process.

In simple words: A document loader reads files and prepares them for AI applications.

Why is it Used?

- Read documents
- Load datasets
- Process files
- Build RAG applications

Flow

File
↓
Document Loader
↓
LangChain Document

Remember

Document Loader = Reads files into LangChain

---

2. Text Loader

Definition

A **Text Loader** loads data from **.txt** files.

Supported File

notes.txt

Uses

- Notes
- Articles
- Logs
- Plain text

Remember

Text Loader = .txt files

---

3. PDF Loader

Definition

A **PDF Loader** extracts text from **PDF documents**.

Supported File

book.pdf

Uses

- Books
- Research papers
- Reports
- Invoices

Remember

PDF Loader = PDF documents

---

4. CSV Loader

Definition

A **CSV Loader** loads data from **CSV (Comma-Separated Values)** files.

Supported File

employees.csv

Uses

- Tabular data
- Sales reports
- Student records
- Datasets

Remember

CSV Loader = Spreadsheet data

---

5. Word Loader

Definition

A **Word Loader** reads Microsoft Word documents.

Supported File

report.docx

Uses

- Reports
- Assignments
- Office documents

Remember

Word Loader = DOCX files

---

6. HTML Loader

Definition

An **HTML Loader** extracts text from HTML pages.

Supported File

index.html

Uses

- Website content
- Web pages
- HTML documents

Remember

HTML Loader = HTML pages

---

7. Web Loader

Definition

A **Web Loader** loads content directly from websites using their URLs.

Example

https://example.com

↓

Extract webpage content.

Uses

- News articles
- Blogs
- Documentation
- Websites

Remember

Web Loader = Website content

---

8. JSON Loader

Definition

A **JSON Loader** reads data stored in **JSON format**.

Example

```
{
  "name": "Rahul",
  "age": 22
}
```

Uses

- APIs
- Configuration files
- Structured datasets

Remember

JSON Loader = JSON data

---

9. Directory Loader

Definition

A **Directory Loader** loads all supported files from a folder.

Example

Folder

```
Documents/
├── notes.txt
```

```
├── report.pdf
├── data.csv
└── resume.docx
```

All files are loaded together.

Advantages

- Bulk loading
- Saves time
- Easy data ingestion

Remember

Directory Loader = Entire folder

---

10. Markdown Loader

Definition

A **Markdown Loader** loads **Markdown (.md)** files.

Supported File

README.md

Uses

- Documentation
- GitHub README files
- Notes

Remember

Markdown Loader = .md files

---

11. Custom Loaders

Definition

A **Custom Loader** is a document loader created by the developer to load unsupported or specialized file formats.

Why is it Used?

- Proprietary file formats
- Database exports
- Internal company documents
- Custom business data

Example

A company stores data in:

employee.xyz

A custom loader reads this file and converts it into LangChain documents.

Advantages

- Fully customizable
- Supports any data source
- Flexible integration

Remember

Custom Loader = Developer-created document loader

Module 10: Text Splitters

---

1. Why Text Splitting?

Definition

Text Splitting is the process of breaking a large document into smaller pieces called **chunks** before sending them to an LLM.

In simple words: Instead of giving the entire document to the AI, we divide it into small, manageable parts.

Why is it Needed?

- LLMs have token limits.
- Large documents cannot be processed at once.
- Improves retrieval accuracy.
- Makes RAG applications more efficient.

Example

Original Document

100-page PDF

↓

After Splitting

Chunk 1
Chunk 2
Chunk 3
Chunk 4
...

Remember

Text Splitting = Large Document → Small Chunks

---

2. Character Text Splitter

Definition

A **Character Text Splitter** divides text after a fixed number of characters.

Example

Chunk Size = 100 characters

Chunk 1 → First 100 characters

Chunk 2 → Next 100 characters

Advantages

- Simple
- Fast
- Easy to use

Disadvantages

- May split sentences in the middle.

Remember

Character Splitter = Split by character count

---

3. Recursive Character Text Splitter

Definition

A **Recursive Character Text Splitter** tries to split text at natural boundaries such as paragraphs, lines, or sentences before splitting by characters.

Priority

Paragraph
↓
Line
↓
Sentence
↓
Word
↓
Character

Advantages

- Better readability
- Preserves context
- Most commonly used in RAG

Remember

Recursive Splitter = Smart text splitting

---

4. Token Text Splitter

Definition

A **Token Text Splitter** divides text based on the number of **tokens** instead of characters.

Why is it Used?

LLMs process **tokens**, not characters.

Example

Chunk Size = 500 Tokens

Each chunk contains approximately 500 tokens.

Advantages

- Matches LLM token limits
- More accurate than character splitting

Remember

Token Splitter = Split by tokens

---

5. Markdown Splitter

Definition

A **Markdown Splitter** splits Markdown files based on headings and sections.

Example

Introduction

Installation

Examples

Conclusion

Each heading becomes a separate chunk.

Uses

- Documentation
- README files
- Notes

Remember

Markdown Splitter = Split by headings

---

6. HTML Splitter

Definition

An **HTML Splitter** divides HTML documents using HTML tags.

Example

<h1>Introduction</h1>

Content...

<h2>Installation</h2>

Each section becomes a chunk.

Uses

- Websites
- HTML documents
- Web scraping

Remember

HTML Splitter = Split by HTML tags

---

7. Code Splitter

Definition

A **Code Splitter** divides source code into logical sections like classes, functions, or methods.

Example

```
def add():
    ...

def subtract():
    ...
```

Each function can become a separate chunk.

Uses

- Code search
- AI coding assistants
- Documentation

Remember

Code Splitter = Split by functions/classes

---

8. Semantic Chunking

Definition

Semantic Chunking splits text based on **meaning** instead of fixed size.

In simple words: Related information stays together in one chunk.

Example

Instead of splitting every 500 characters:

Paragraph about AI

Paragraph about ML

Paragraph about NLP

Each topic becomes a separate chunk.

Advantages

- Better retrieval
- Preserves meaning
- Higher RAG accuracy

Remember

Semantic Chunking = Split by meaning

---

9. Chunk Size

Definition

Chunk Size is the maximum amount of text placed in one chunk.

Example

Chunk Size = 500 Tokens

Each chunk contains approximately 500 tokens.

Choosing Chunk Size

Small Chunks

- Faster search
- More precise retrieval

Large Chunks

- More context
- Fewer chunks

Remember

Chunk Size = Size of each chunk

---

10. Chunk Overlap

Definition

Chunk Overlap means repeating a small portion of one chunk in the next chunk.

Example

Without Overlap

Chunk 1
Sentence A
Sentence B

Chunk 2
Sentence C
Sentence D

With Overlap

Chunk 1
Sentence A
Sentence B

Chunk 2
Sentence B
Sentence C
Sentence D

Advantages

- Preserves context
- Better retrieval
- Avoids losing information at chunk boundaries

Remember

Chunk Overlap = Shared text between chunks

Module 11: Embeddings

---

1. What are Embeddings?

Definition

Embeddings are numerical vector representations of text that capture its meaning.

In simple words: Embeddings convert words, sentences, or documents into numbers so computers can compare their meanings.

Why are They Used?

- Semantic search
- RAG applications
- Similarity search
- Recommendation systems
- Clustering

Example

"I love AI"

↓

Embedding

[0.21, -0.45, 0.78, ...]

Remember

Embeddings = Text → Vector (Numbers)

---

2. Embedding Models

Definition

An **Embedding Model** converts text into embedding vectors.

Popular Embedding Models

- OpenAI Embeddings
- Hugging Face Embeddings
- Google Embeddings
- Ollama Embeddings

Uses

- Vector databases
- Similarity search
- Document retrieval
- RAG

Remember

Embedding Model = Creates embeddings

---

3. OpenAI Embeddings

Definition

OpenAI Embeddings generate high-quality vector representations using OpenAI's embedding models.

Advantages

- High accuracy

- Fast
- Excellent semantic understanding
- Easy LangChain integration

Uses

- RAG
- Search
- Chatbots

Remember

OpenAI Embeddings = High-quality cloud embeddings

---

4. Hugging Face Embeddings

Definition

Hugging Face Embeddings are generated using open-source embedding models from Hugging Face.

Advantages

- Free (many models)
- Open source
- Large model selection
- Local execution possible

Uses

- Offline applications
- Research
- Custom AI systems

Remember

Hugging Face = Open-source embeddings

---

5. Google Embeddings

Definition

Google Embeddings generate vector representations using Google's embedding models.

Advantages

- High-quality embeddings
- Good multilingual support
- Cloud-based

Uses

- Search
- RAG
- AI assistants

Remember

Google Embeddings = Google's vector models

---

6. Ollama Embeddings

Definition

Ollama Embeddings generate embeddings locally using models installed with Ollama.

Advantages

- Runs locally
- Privacy
- Offline support
- No cloud dependency

Uses

- Local RAG
- Offline AI
- Secure environments

Remember

Ollama Embeddings = Local vector generation

---

7. Cosine Similarity

Definition

Cosine Similarity measures how similar two embedding vectors are by comparing the angle between them.

Value Range

1 → Exactly the same meaning
0 → Unrelated
-1 → Completely opposite (rare in embeddings)

Uses

- Semantic search
- Document retrieval
- Recommendation systems

Remember

Higher Cosine Similarity = More similar meaning

---

8. Dot Product

Definition

The **Dot Product** measures similarity by multiplying corresponding values of two vectors and summing the results.

Uses

- Vector search
- Ranking
- Retrieval

Key Points

- Larger value → More similar (when vectors are normalized)
- Fast to compute

Remember

Dot Product = Vector similarity score

---

9. Euclidean Distance

Definition

Euclidean Distance measures the straight-line distance between two embedding vectors.

Interpretation

Small Distance → More Similar

Large Distance → Less Similar

Uses

- Clustering
- Similarity search
- Machine Learning

Remember

Smaller Euclidean Distance = More similar vectors

Module 12: Vector Stores

---

1. What is a Vector Store?

Definition

A **Vector Store** is a database that stores **embeddings (vectors)** and allows fast similarity searches.

In simple words: A Vector Store stores document embeddings so the AI can quickly find the most relevant information.

Why is it Used?

- Store embeddings
- Fast document search
- Semantic search
- RAG applications

Flow

Document
↓
Embedding Model
↓

Embedding (Vector)
↓
Vector Store
↓
Similarity Search

Remember

Vector Store = Database for Embeddings

---

2. Chroma

Definition

Chroma is an open-source vector database designed specifically for AI and LangChain applications.

Advantages

- Easy to use
- Open source
- Local storage
- Excellent LangChain support

Uses

- RAG
- Chatbots
- Document search

Remember

Chroma = Simple local vector database

---

3. FAISS

Definition

FAISS (Facebook AI Similarity Search) is an open-source library developed by Meta for fast similarity search.

Advantages

- Extremely fast
- Efficient for large datasets
- Local execution
- Free

Uses

- Semantic search
- Recommendation systems
- Large-scale retrieval

Remember

FAISS = Fast similarity search

---

4. Pinecone

Definition

Pinecone is a cloud-based managed vector database.

Advantages

- Fully managed
- Highly scalable
- Real-time search
- No infrastructure management

Uses

- Production AI applications
- Enterprise RAG
- Large document collections

Remember

Pinecone = Cloud vector database

---

5. Weaviate

Definition

Weaviate is an open-source vector database that supports semantic search and AI applications.

Advantages

- Open source
- GraphQL support
- Scalable
- Hybrid search support

Uses

- Knowledge bases
- Enterprise search
- RAG systems

Remember

Weaviate = AI-powered vector database

---

6. Qdrant

Definition

Qdrant is an open-source vector database optimized for semantic search and recommendations.

Advantages

- Fast search
- Filtering support
- Open source
- Scalable

Uses

- Recommendation systems
- AI search
- Chatbots

Remember

Qdrant = Fast semantic search

---

7. Milvus

Definition

Milvus is an open-source vector database built for handling very large collections of embeddings.

Advantages

- Highly scalable
- Distributed architecture
- High performance
- Enterprise-ready

Uses

- Large AI systems
- Big data search
- Enterprise RAG

Remember

Milvus = Large-scale vector database

---

8. PGVector

Definition

PGVector is a PostgreSQL extension that adds vector storage and similarity search to PostgreSQL.

Advantages

- Uses PostgreSQL
- Easy integration
- SQL support
- No separate vector database needed

Uses

- Existing PostgreSQL projects
- RAG
- AI applications

Remember

PGVector = Vectors inside PostgreSQL

---

9. Storing Embeddings

Definition

Storing Embeddings means saving embedding vectors in a vector database for future retrieval.

Flow

Document
↓
Embedding Model
↓
Vector
↓
Vector Store

Why is it Needed?

- Avoid generating embeddings repeatedly
- Faster search
- Efficient retrieval

Remember

Store Once → Search Many Times

---

10. Similarity Search

Definition

Similarity Search finds documents whose embeddings are most similar to the user's query embedding.

Example

Query:

"What is LangChain?"

↓

Vector Store returns:

Document 1 □
Document 3 □
Document 8 □

Most relevant documents are returned.

Uses

- RAG
- Search engines
- Question answering
- Recommendation systems

Remember

Similarity Search = Find similar documents using embeddings

Module 13: Retrievers

---

1. What is a Retriever?

Definition

A **Retriever** searches a knowledge base or vector store and returns the **most relevant documents** for a user's query.

In simple words: A retriever finds the best documents before the AI generates an answer.

Flow

User Question
↓
Retriever
↓
Relevant Documents
↓
LLM
↓
Answer

Why is it Used?

- Improve answer accuracy
- Reduce hallucinations
- Retrieve relevant information
- Build RAG applications

Remember

Retriever = Finds relevant documents

---

2. Vector Store Retriever

Definition

A **Vector Store Retriever** searches a vector database using embeddings.

Flow

Question
↓
Embedding
↓
Vector Store
↓
Top Matching Documents

Uses

- RAG
- Semantic search
- Chatbots

Remember

Vector Store Retriever = Embedding-based search

---

3. Multi Query Retriever

Definition

A **Multi Query Retriever** creates **multiple versions of the same question** and searches using all of them.

Example

Original Question

What is AI?

Generated Queries

Explain AI

Define Artificial Intelligence

What does AI mean?

Results from all queries are combined.

Advantages

- Better coverage
- Higher retrieval accuracy
- Finds more relevant documents

Remember

Multi Query Retriever = One question → Multiple searches

---

4. Self Query Retriever

Definition

A **Self Query Retriever** lets the LLM understand the user's question and automatically create a search query with filters.

Example

User asks:

Find AI books published after 2022.

Retriever understands:

- Topic = AI
- Year > 2022

Then searches accordingly.

Advantages

- Smart filtering
- Better search
- Less manual work

Remember

Self Query Retriever = AI creates the search query

---

5. Parent Document Retriever

Definition

A **Parent Document Retriever** retrieves a small matching chunk but returns the larger parent document for better context.

Flow

Large Document
↓
Split into Chunks
↓
Match One Chunk
↓
Return Parent Document

Advantages

- More context
- Better answers
- Preserves document meaning

Remember

Parent Retriever = Retrieve chunk, return full context

---

6. Multi Vector Retriever

Definition

A **Multi Vector Retriever** stores **multiple embeddings** for the same document to improve retrieval.

Example

One document may have embeddings for:

- Summary

- Title
- Content

Search checks all of them.

Advantages

- Better accuracy
- Flexible retrieval
- Improved search quality

Remember

Multi Vector Retriever = Multiple embeddings per document

---

7. Contextual Compression Retriever

Definition

A **Contextual Compression Retriever** retrieves documents and then removes unnecessary information before sending them to the LLM.

Flow

Retrieve Document
↓
Remove Irrelevant Content
↓
Send Important Information

Advantages

- Fewer tokens
- Lower cost
- Better focus

Remember

Compression Retriever = Keep only relevant content

---

8. Ensemble Retriever

Definition

An **Ensemble Retriever** combines the results of **multiple retrievers** to improve retrieval quality.

Example

Vector Retriever
+
Keyword Retriever
+
BM25 Retriever
↓
Combined Results

Advantages

- Better accuracy
- More reliable
- Covers different search methods

Remember

Ensemble Retriever = Combine multiple retrievers

---

9. MMR Search

Definition

MMR (Maximal Marginal Relevance) retrieves documents that are **both relevant and diverse**, avoiding duplicate or very similar results.

Example

Instead of returning:

Document A
Document A (similar)
Document A (similar)

MMR returns:

Document A
Document B
Document C

Advantages

- Diverse results
- Less redundancy
- Better context

Remember

MMR = Relevant + Diverse documents

---

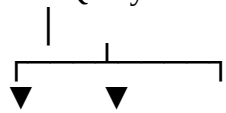
10. Hybrid Search

Definition

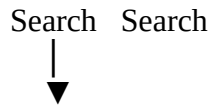
Hybrid Search combines **keyword search** and **vector (semantic) search**.

Flow

User Query



Keyword Vector



Advantages

- Higher accuracy
- Better recall
- Handles both exact words and semantic meaning

Example

Searching for:

car

Can also retrieve documents containing:

automobile
vehicle

because of semantic search.

Remember

Hybrid Search = Keyword Search + Vector Search

Module 14: Retrieval-Augmented Generation (RAG)

---

1. Introduction to RAG

Definition

RAG (Retrieval-Augmented Generation) is a technique where an LLM retrieves relevant information from external data before generating a response.

In simple words: Instead of answering only from its training data, the AI first searches your documents and then answers.

Why is it Used?

- Answer questions using your own data
- Reduce hallucinations
- Improve accuracy
- Keep information up to date

Flow

User Question
↓
Retriever
↓
Relevant Documents
↓
LLM
↓
Final Answer

Example

Question:

What is LangChain?

AI searches your PDF or database first, then answers using that information.

Remember

RAG = Retrieve → Generate

---

2. RAG Architecture

Definition

RAG Architecture shows how all components work together to answer a user's question.

Architecture

User
↓
Retriever
↓
Vector Store
↓
Relevant Documents
↓
LLM
↓
Final Answer

Main Components

- User
- Retriever
- Vector Store
- LLM
- Response

Remember

Retriever + Vector Store + LLM = RAG

---

3. Indexing Pipeline

Definition

The **Indexing Pipeline** prepares documents before they are searched.

Steps

Documents
↓
Load Documents

↓
Split into Chunks
↓
Create Embeddings
↓
Store in Vector Database

Why is it Needed?

- Faster retrieval
- Better search
- Organized storage

Remember

Indexing = Prepare documents for search

---

4. Retrieval Pipeline

Definition

The **Retrieval Pipeline** finds the most relevant document chunks for a user's query.

Flow

User Question
↓
Embedding
↓
Vector Store
↓
Top Matching Chunks

Advantages

- Fast search
- Relevant context
- Better answers

Remember

Retrieval = Find relevant information

---

5. Generation Pipeline

Definition

The **Generation Pipeline** generates the final answer using the retrieved documents.

Flow

Retrieved Documents
↓
Prompt
↓
LLM
↓
Answer

Why is it Important?

- Uses real information
- Improves response quality
- Reduces incorrect answers

Remember

Generation = LLM creates the answer

---

6. Context Injection

Definition

Context Injection means adding the retrieved documents into the prompt before sending it to the LLM.

Example

Question:
What is LangChain?

Context:
LangChain is an open-source framework...

Answer:
LangChain is...

Advantages

- More accurate answers
- Uses external knowledge
- Better context understanding

Remember

Context Injection = Add retrieved data to the prompt

---

7. Source Documents

Definition

Source Documents are the original documents from which the retrieved information comes.

Examples

- PDF
- Word Document
- Website
- CSV File
- Database

Why are They Important?

- Show where the answer came from
- Increase trust
- Allow users to verify information

Remember

Source Documents = Original data source

---

8. Conversational RAG

Definition

Conversational RAG combines RAG with conversation memory so the AI remembers previous messages while retrieving new information.

Example

User: Explain LangChain.

User: Who created it?

AI remembers "it" refers to LangChain.

Advantages

- Maintains context
- Better conversations
- More natural responses

Remember

Conversational RAG = RAG + Memory

---

9. Advanced RAG Concepts

Definition

Advanced RAG improves retrieval quality and response accuracy using advanced retrieval techniques.

Common Techniques

- Multi Query Retrieval
- Hybrid Search
- Context Compression
- Parent Document Retrieval
- Re-ranking Results
- Metadata Filtering

Benefits

- Better retrieval
- Higher accuracy
- Lower hallucinations
- Faster search

Remember

Advanced RAG = Smarter retrieval techniques

Module 15: Tools

---

1. What is a Tool?

Definition

A **Tool** is a function or external service that an LLM can use to perform tasks beyond text generation.

In simple words: A tool gives the AI extra abilities like searching the web, doing calculations, or calling APIs.

Why is it Used?

- Search information
- Perform calculations
- Access databases
- Call APIs
- Automate tasks

Example

User: What's the weather today?

AI
↓
Weather Tool
↓
Current Weather
↓
Answer

Remember

Tool = Extra capability for the AI

---

2. Built-in Tools

Definition

Built-in Tools are tools already provided by LangChain or supported integrations.

Examples

- Web Search
- Calculator
- Python Execution
- Database Query

Advantages

- Ready to use
- Easy integration
- Saves development time

Remember

Built-in Tools = Predefined tools

---

3. Custom Tools

Definition

A **Custom Tool** is created by the developer to perform application-specific tasks.

Example

Calculate Student GPA

OR

Check Bank Balance

OR

Book a Ticket

Advantages

- Flexible
- Business-specific

- Fully customizable

Remember

Custom Tool = Developer-created tool

---

4. @tool Decorator

Definition

The **@tool decorator** converts a Python function into a LangChain tool.

Example

```
from langchain.tools import tool

@tool
def add(a: int, b: int):
    return a + b
```

Now the AI can use the add() function as a tool.

Advantages

- Easy tool creation
- Cleaner code
- Automatic tool registration

Remember

@tool = Convert function into a tool

---

5. Structured Tools

Definition

A **Structured Tool** accepts multiple typed inputs using a defined schema.

Example

Input

City = Pune

Date = Tomorrow

Instead of a single text string.

Advantages

- Input validation
- Multiple parameters
- Better reliability

Remember

Structured Tool = Tool with structured inputs

---

6. Tool Calling

Definition

Tool Calling is the process where the LLM decides that it needs a tool, calls it, and uses the returned result to answer the user.

Flow

User Question
↓
LLM

↓
Choose Tool
↓
Execute Tool
↓
Tool Result
↓
Final Answer

Example

User:
Calculate 45 × 12.

LLM

Calculator Tool

540

Remember

Tool Calling = AI selects and uses a tool

---

7. Function Calling

Definition

Function Calling allows an LLM to call predefined functions with structured arguments instead of generating plain text.

Example

Function

get_weather(city)

User

Weather in Mumbai

LLM calls

get_weather("Mumbai")

Advantages

- Accurate inputs
- Structured data
- Easy API integration

Remember

Function Calling = LLM calls predefined functions

---

8. Multiple Tools

Definition

Multiple Tools means an AI application can use more than one tool to solve a user's request.

Example

User:
Find today's weather and convert 30°C to Fahrenheit.

↓

Weather Tool
+
Temperature Converter
↓

Final Answer

Advantages

- Handles complex tasks
- More capable AI

- Better automation

Remember

Multiple Tools = AI uses several tools together

Module 16: Agents

---

1. What are Agents?

Definition

An **Agent** is an AI system that can **think, decide, and use tools** to complete a task.

In simple words: An Agent doesn't just answer questions—it decides what actions to take before giving the final answer.

Why is it Used?

- Use tools automatically
- Solve complex tasks
- Perform multi-step reasoning
- Automate workflows

Example

User:
What is today's weather in Pune?

↓

Agent
↓
Uses Weather Tool
↓
Gets Result
↓
Returns Answer

Remember

Agent = Think + Use Tools + Answer

---

2. Agent Architecture

Definition

Agent Architecture shows how an agent processes a request and interacts with tools.

Architecture

User
↓
LLM
↓
Reasoning
↓
Select Tool
↓
Execute Tool
↓
Observation
↓
Final Answer

Main Components

- User Input
- LLM
- Tools
- Reasoning
- Final Response

Remember

Agent = LLM + Tools + Reasoning

---

3. ReAct Pattern

Definition

ReAct (Reason + Act) is an agent pattern where the AI first **reasons**, then **takes an action**, observes the result, and repeats if needed.

Flow

Question
↓
Think
↓
Use Tool
↓
Observe
↓
Think Again
↓
Final Answer

Advantages

- Better reasoning
- Uses tools intelligently
- Solves complex problems

Remember

ReAct = Reason → Act → Observe

---

4. Tool Calling Agent

Definition

A **Tool Calling Agent** automatically decides **which tool** to use based on the user's request.

Example

User:
What is 50 × 12?

↓

Agent

↓

Calculator Tool

↓

600

Advantages

- Automatic tool selection
- Faster execution
- Flexible workflows

Remember

Tool Calling Agent = Chooses the correct tool automatically

---

5. OpenAI Tools Agent

Definition

An **OpenAI Tools Agent** is an agent that uses OpenAI models with tool/function calling capabilities.

Uses

- API calls

- Web search
- Database queries
- Custom functions

Advantages

- Native tool support
- Reliable execution
- Easy integration

Remember

OpenAI Tools Agent = OpenAI model + Tools

---

6. Agent Executor

Definition

The **Agent Executor** manages the complete execution of an agent.

Responsibilities

- Run the agent
- Execute tools
- Handle intermediate steps
- Return the final response

Flow

Agent
↓
Executor
↓
Tool Calls
↓
Final Output

Remember

Agent Executor = Runs the agent

---

7. Agent Scratchpad

Definition

The **Agent Scratchpad** is the agent's temporary workspace where it stores its intermediate reasoning and tool results.

Example

Thought:
Need weather information.

↓

Action:
Call Weather Tool.

↓

Observation:
30°C

↓

Final Answer:
Today's temperature is 30°C.

Advantages

- Tracks reasoning
- Stores intermediate steps
- Helps with multi-step tasks

Remember

Scratchpad = Agent's temporary notes

---

8. Multi-step Reasoning

Definition

Multi-step Reasoning means solving a problem through multiple logical steps instead of one direct answer.

Example

Question
↓
Find Data
↓
Calculate
↓
Analyze
↓
Final Answer

Uses

- Data analysis
- Planning
- Complex reasoning
- Decision making

Remember

Multi-step Reasoning = Solve step by step

---

9. Agent Planning

Definition

Agent Planning is the process of deciding the sequence of actions needed to complete a task.

Example

Task:
Book a Flight

↓

Search Flights

↓

Compare Prices

↓

Choose Best Option

↓

Book Ticket

Advantages

- Organized execution
- Better decision making
- Efficient workflows

Remember

Planning = Decide the steps first

---

10. Agent Error Handling

Definition

Agent Error Handling is the process of detecting and managing errors during tool execution or reasoning.

Example

Tool Error

↓

Retry

↓

Alternative Tool

↓

Inform User

Common Errors

- Tool failure
- Invalid input
- API timeout
- Network issues

Remember

Error Handling = Detect, recover, and continue

Module 17: Message History

---

1. Chat Message History

Definition

Chat Message History stores the conversation between the user and the AI.

In simple words: It keeps track of previous messages so the AI can respond with context.

Example

User: Hi

AI: Hello!

User: What is LangChain?

AI: LangChain is...

Advantages

- Maintains context
- Natural conversations
- Better responses

Remember

Chat Message History = Conversation record

---

2. Session Management

Definition

Session Management keeps conversations separate for different users or different chat sessions.

Example

Session 1

User A
Conversation A

Session 2

User B
Conversation B

Each user has an independent conversation.

Advantages

- Separate chat history
- Multi-user support
- Personalized conversations

Remember

Session Management = One history per session

---

3. Persistent History

Definition

Persistent History stores chat history permanently so it is available even after restarting the application.

Example

Today

↓

Save Messages

↓

Tomorrow

↓

Load Previous Conversation

Storage Options

- Database
- File
- Cloud Storage

Advantages

- Long-term memory
- Resume conversations
- Better user experience

Remember

Persistent History = Saved permanently

---

4. Redis Chat History

Definition

Redis Chat History stores conversation history in a **Redis database**.

Why is it Used?

- Very fast
- In-memory storage
- Suitable for real-time applications

Uses

- Chatbots
- AI assistants
- Live messaging systems

Remember

Redis = Fast chat history storage

---

5. SQL Chat History

Definition

SQL Chat History stores conversations in a **SQL database** such as PostgreSQL or MySQL.

Advantages

- Permanent storage
- Structured data
- Easy querying
- Reliable backups

Uses

- Enterprise AI
- Customer support
- CRM systems

Remember

SQL Chat History = Database-based conversation storage

Module 18: Streaming

---

1. Token Streaming

Definition

Token Streaming sends the LLM's response **one token (small piece of text)** at a time instead of waiting for the complete response.

In simple words: The AI starts showing the answer as it is being generated.

Example

Without Streaming

Hello! How can I help you today?

With Token Streaming

Hello...
How...
can...
I...
help...
you...
today?

Advantages

- Faster user experience
- Real-time responses
- Better for chat applications

Remember

Token Streaming = Token-by-token output

---

2. Response Streaming

Definition

Response Streaming sends the response gradually in larger chunks until the complete answer is generated.

Flow

LLM
↓
Chunk 1
↓
Chunk 2
↓
Chunk 3
↓
Complete Response

Uses

- Chatbots
- AI assistants
- Long responses

Remember

Response Streaming = Chunk-by-chunk output

---

3. Event Streaming

Definition

Event Streaming streams different events that occur while the application is running.

Examples of Events

- Tool Started
- Tool Finished
- Model Started
- Model Finished
- Error Occurred

Flow

Start

↓
Model Running
↓
Tool Called
↓
Tool Finished
↓
Final Response

Advantages

- Easy debugging
- Monitor workflow
- Track execution

Remember

Event Streaming = Stream application events

---

4. Async Streaming

Definition

Async Streaming combines asynchronous execution with streaming so the application remains responsive while generating output.

Advantages

- Better performance
- Non-blocking execution
- Faster user interaction

Example

User Request
↓
Async Processing

↓
Streaming Response

Remember

Async Streaming = Streaming + Asynchronous execution

Module 19: Callbacks

---

1. Callback System

Definition

A **Callback System** lets you execute custom code when specific events occur during LangChain execution.

In simple words: A callback is triggered automatically when something happens.

Example

Model Starts
↓
Callback Runs
↓
Log Information

Uses

- Logging
- Monitoring
- Debugging
- Analytics

Remember

Callback = Run code on events

---

2. Logging

Definition

Logging records important information about application execution.

Example

10:30 AM

Model Started

10:30 AM

Response Generated

Advantages

- Debug errors
- Track execution
- Monitor performance

Remember

Logging = Record application activity

---

3. Token Usage

Definition

Token Usage tracks how many input and output tokens are used by an LLM.

Why is it Important?

- Monitor API usage

- Estimate cost
- Optimize prompts

Example

Input Tokens : 150

Output Tokens : 250

Total Tokens : 400

Remember

Token Usage = Count input and output tokens

---

4. Monitoring

Definition

Monitoring continuously observes the performance and health of an AI application.

What Can Be Monitored?

- Response time
- Errors
- Token usage
- Tool execution
- API requests

Advantages

- Improve reliability
- Detect problems
- Optimize performance

Remember

Monitoring = Observe application performance

---

5. Custom Callbacks

Definition

A **Custom Callback** is a callback created by the developer for specific application needs.

Example

Model Completes

↓

Save Response to Database

↓

Notify User

Advantages

- Flexible
- Custom behavior
- Business-specific logic

Remember

Custom Callback = Developer-defined callback

Module 20: Structured Output

---

1. JSON Schema

Definition

A **JSON Schema** defines the expected structure of a JSON response.

In simple words: It tells the AI exactly how the output should be formatted.

Example

```
{
  "name": "Rahul",
  "age": 22
}
```

Advantages

- Consistent output
- Easy to parse
- Standardized format

Remember

JSON Schema = Define JSON structure

---

2. Pydantic Models

Definition

Pydantic Models define structured data using Python classes with type validation.

Example

```
class Student:
    name: str
    age: int
```

Advantages

- Automatic validation
- Type checking
- Cleaner code

Remember

Pydantic = Python model with validation

---

3. Typed Responses

Definition

A **Typed Response** ensures the AI returns data with predefined data types.

Example

Name → String

Age → Integer

Marks → Float

Advantages

- Predictable output
- Fewer errors
- Easy integration

Remember

Typed Response = Output with fixed data types

---

4. Validation

Definition

Validation checks whether the generated output follows the required structure and data types.

Example

Expected

Age = Integer

Generated

Age = "Twenty"

Validation fails because the value is not an integer.

Advantages

- Improves reliability
- Prevents invalid data
- Reduces application errors

Remember

Validation = Verify output correctness

---

5. Error Handling

Definition

Error Handling manages situations where the generated output is invalid or does not match the expected structure.

Example

Invalid Output

↓

Validation Error

↓

Retry or Fix Output

↓

Valid Response

Common Errors

- Invalid JSON
- Missing fields
- Wrong data type
- Incorrect format

Remember

Error Handling = Detect and fix output errors

Module 21: LangSmith

---

1. Introduction to LangSmith

Definition

LangSmith is a developer platform for **debugging, testing, evaluating, and monitoring** LangChain and LLM applications.

In simple words: LangSmith helps you understand how your AI application works and identify issues.

Why is it Used?

- Debug AI applications
- Monitor performance
- Evaluate responses

- Test workflows
- Improve reliability

Flow

User Request
↓
LangChain App
↓
LangSmith
↓
Trace • Debug • Monitor • Evaluate

Remember

LangSmith = Debug + Test + Monitor AI Apps

---

2. Tracing

Definition

Tracing records every step an AI application performs while processing a request.

Example

User Question
↓
Prompt
↓
LLM
↓
Tool Call
↓
Final Answer

LangSmith records each step.

Advantages

- Easy debugging
- Complete execution history
- Better understanding of workflows

Remember

Tracing = Record every execution step

---

3. Debugging

Definition

Debugging is the process of finding and fixing errors in an AI application.

Example

Tool Failed

↓

Check Trace

↓

Find Error

↓

Fix Problem

Advantages

- Faster issue detection
- Easier troubleshooting
- Better application reliability

Remember

Debugging = Find and fix errors

---

4. Evaluation

Definition

Evaluation measures how well an AI application performs.

What Can Be Evaluated?

- Accuracy
- Response quality
- Relevance
- Correctness

Advantages

- Compare models
- Improve prompts
- Measure application quality

Remember

Evaluation = Measure AI performance

---

5. Monitoring

Definition

Monitoring continuously tracks the health and performance of an AI application.

What Can Be Monitored?

- Response time
- Errors
- Token usage
- Tool execution
- API performance

Advantages

- Detect problems early
- Improve reliability
- Optimize performance

Remember

Monitoring = Observe application performance

---

6. Dataset Testing

Definition

Dataset Testing evaluates an AI application using a predefined set of test questions and expected results.

Example

Test Dataset
↓
Run AI Application
↓
Compare Results
↓
Evaluation Report

Advantages

- Consistent testing
- Measure improvements

- Prevent regressions

Remember

Dataset Testing = Test AI using predefined datasets

Module 22: Production Best Practices

---

1. Project Structure

Definition

A **Project Structure** is the organized arrangement of folders and files in an application.

Example

```
project/
├── app/
├── prompts/
├── tools/
├── models/
├── utils/
├── config/
└── main.py
```

Advantages

- Cleaner code
- Easy maintenance
- Better teamwork

Remember

Good Structure = Easy maintenance

---

2. Environment Variables

Definition

Environment Variables store sensitive information outside the source code.

Examples

- API Keys
- Database URL
- Secret Keys

Example File

.env

OPENAI_API_KEY=xxxxxxxx
DATABASE_URL=xxxxxxxx

Advantages

- Better security
- Easy configuration
- Avoid hardcoding secrets

Remember

Environment Variables = Store secrets safely

---

3. Configuration Management

Definition

Configuration Management keeps application settings in one place for easy management.

Examples

- API URLs
- Model names
- Token limits
- Database settings

Advantages

- Easy updates
- Better organization
- Environment-specific settings

Remember

Configuration = Centralized settings

---

4. Logging

Definition

Logging records important events while the application runs.

Example

10:00 AM

Application Started

10:01 AM

Request Completed

Advantages

- Debugging
- Monitoring
- Performance tracking

Remember

Logging = Record application events

---

5. Retry Mechanisms

Definition

A **Retry Mechanism** automatically retries an operation if it fails.

Example

API Request

↓

Failed

↓

Retry

↓

Success

Advantages

- Handles temporary failures
- Improves reliability
- Reduces manual intervention

Remember

Retry = Try again after failure

---

6. Rate Limiting

Definition

Rate Limiting restricts how many requests can be made within a certain period.

Why is it Used?

- Prevent API overuse
- Avoid service limits
- Protect applications

Example

Limit

100 Requests / Minute

Remember

Rate Limiting = Control request frequency

---

7. Caching

Definition

Caching stores frequently used data temporarily to avoid repeated processing.

Example

First Request

↓

Generate Response

↓

Save in Cache

↓

Next Request

↓

Return Cached Response

Advantages

- Faster responses
- Lower API cost
- Better performance

Remember

Caching = Store results for reuse

---

8. Error Handling

Definition

Error Handling detects and manages errors without crashing the application.

Example

API Error

↓

Catch Error

↓

Show Friendly Message

↓

Continue Running

Common Errors

- API failures
- Invalid input
- Network issues
- Database errors

Remember

Error Handling = Handle errors gracefully

---

9. Security Best Practices

Definition

Security Best Practices protect the application, users, and sensitive information.

Best Practices

- Keep API keys secret
- Validate user input
- Use HTTPS
- Apply authentication
- Restrict permissions
- Update dependencies regularly

Advantages

- Protects data

- Prevents attacks
- Improves reliability

Remember

Security = Protect data and applications

---

10. Performance Optimization

Definition

Performance Optimization improves the speed and efficiency of an AI application.

Techniques

- Caching
- Streaming
- Async execution
- Efficient prompts
- Optimized retrieval

Advantages

- Faster responses
- Lower costs
- Better user experience

Remember

Performance Optimization = Faster and more efficient application

---

11. Deployment

Definition

Deployment is the process of making an AI application available for users.

Common Deployment Platforms

- Local Server
- Docker
- Cloud (AWS, Azure, GCP)
- Render
- Railway
- Vercel (Frontend)

Deployment Flow

Develop
↓
Test
↓
Deploy
↓
Users Access the Application

Best Practices

- Test before deployment
- Monitor the application
- Secure environment variables
- Enable logging
- Keep backups

Remember

Deployment = Make the application available to users